Why Heterogeneous GPU Serving Needs Phase-Aware Scheduling

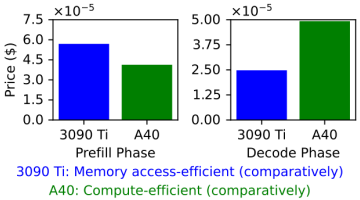


Figure 1. Prefill and decode prices for a single request with input and output lengths of 512 and 16 on 3090Ti and A40.

Observation

- Prefill: compute-intensive.
- Decode: memory-bandwidth-intensive.

A40: 149.7 FP16 TFLOPS. 3090 Ti: 1008 GB/s. Table 1.

Challenge

- Heterogeneous GPU resources.
- Limited and heterogeneous network bandwidth.
- KV cache transfer overhead.

Research question

How to jointly optimize GPU allocation, phase placement, and communication?

The better GPU can differ by phase. KV transfer makes a FLOPS-only or bandwidth-only split insufficient.

Source: ThunderServe Sec. 1, Fig. 1, Table 1. Prices in the figure are per request.

Joint Optimization of GPU Placement, Parallelism, and Routing

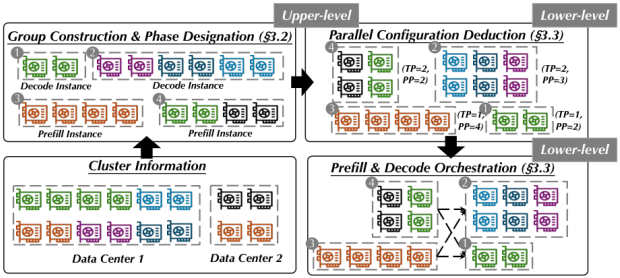


Figure 3. Workflow of our scheduling algorithm.

UPPER-LEVEL — TABU SEARCH

- GPU group construction.
- Prefill / decode phase designation.

LOWER-LEVEL — CONFIGURATION

- TP / PP configuration.
- Prefill / decode request routing.
- Network-aware KV cache transfer.

Additional mechanisms: KV cache compression and lightweight rescheduling.

Upper level assigns groups and phases. Lower level sets parallelism and routing. Together they score one deployment plan.

Source: ThunderServe Sec. 3.2–3.4, Fig. 3.

Does Phase-Aware Scheduling Improve Serving Efficiency?

EXPERIMENT 1 — TABLE 3

Workload-aware deployment

Coding: **8 prefill / 4 decode**

Conversation: **4 prefill / 8 decode**

LLaMA-30B, same 32-GPU cloud pool. Coding has longer prompts; conversation has longer outputs.

EXPERIMENT 2 — FIGURE 9

	Coding	Conv.
vs. HexGen	1.5×	1.3×
vs. DistServe	1.5×	2.1×

HexGen: heterogeneous cloud GPUs.
DistServe: homogeneous 8×A100, similar price. Not the same machines.

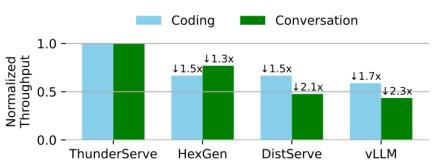


Figure 9. Throughput scaled by ThunderServe.

vLLM, the fourth group, is also in-house. HexGen and DistServe are different hardware setups.

Workload-aware placement improves overall serving throughput under heterogeneous resource constraints.

Source: ThunderServe Sec. 5.1–5.3, Fig. 9, Table 3. Relative throughput is ThunderServe over each baseline.

Phase-Aware Deployment Improves Cost Efficiency

1. JOINT OPTIMIZATION

One deployment plan

GPU grouping, phase designation, TP/PP, and request routing.

2. PERFORMANCE

1.7× average, up to 2.1×

Overall throughput claim in the paper. Not one bar of Figure 9.

3. DYNAMIC ADAPTATION

Full 157 s, lightweight 13 s

Table 4 overall time. The gap includes model reload, not search time alone.

Table 4 breakdown. Full: rescheduling $54(\pm5)$ s + reload $103(\pm10)$ s = $157(\pm13)$ s. Lightweight: rescheduling $13(\pm2)$ s + reload 0 s = $13(\pm2)$ s.

ThunderServe demonstrates that heterogeneous GPU serving benefits from jointly optimizing inference phases, hardware allocation, and communication.

ThunderServe focuses on phase-level resource allocation; finer-grained workload partitioning is another possible direction.

Source: Abstract and Sec. 6 for the 1.7×/2.1× claim. Sec. 5.4 and Table 4 for rescheduling time.